

03 · Estrategias

Quant-Backtester · material de estudio

En este capítulo escribes las tres estrategias del proyecto. Cada una es una función que recibe una tabla de precios de cierre (fechas × activos) y devuelve una tabla de señales de la misma forma, con `-1` (corto), `0` (fuera) o `1` (largo). Dos siguen la tendencia (cruce de medias móviles y momentum de 12 meses) y la tercera apuesta por la vuelta a la media (bandas de Bollinger). Son hipótesis opuestas sobre cómo se mueven los precios, y por eso conviene probarlas en el mismo banco de pruebas.

Por el camino aprenderás tres técnicas que reaparecen en cualquier backtest vectorizado: ventanas móviles con calentamiento, cambio de frecuencia (decidir a fin de mes y aplicar a diario) y máquinas de estado sin bucles. El capítulo cubre los TODO 3.1 `sma_crossover`, 3.2 `momentum_12m` y 3.3 `bollinger_mean_reversion`, en `src/strategies.py`. Sus tests (`tests/test_strategies.py`) no dependen del motor del capítulo 02; el motor solo lo necesitarás para backtestear las señales en el notebook 02.

1. Intuición

Una estrategia es una función precios → señales

El motor no sabe nada de medias ni de bandas: recibe una tabla de señales y la convierte en posiciones, rendimientos y costes. Para que sirva con cualquier estrategia, todas cumplen el mismo contrato:

- Entrada: un DataFrame de cierres limpios, con índice de fechas (`DatetimeIndex`) ordenado y sin repetir, y una columna por activo. Los parámetros llegan como argumentos con nombre.
- Salida: un DataFrame con exactamente el mismo índice y las mismas columnas, en el mismo orden, sin NaN y con valores `-1`, `0` o `1`.
- Causalidad: la señal del día `t` solo puede usar precios hasta `t`, incluido.
- Sin desfase: la estrategia no hace `shift` de su señal; eso lo hace el motor.

El decorador `@strategy`, ya escrito, hace cumplir las dos primeras reglas. Antes de llamar a tu función revisa los precios (`_check_prices`): DataFrame, índice `DatetimeIndex` ordenado y sin fechas repetidas, tabla no vacía. Después revisa tu resultado (`_check_signals`): que sea un DataFrame y no una Series (`TypeError`), que el índice sea idéntico al de los precios (el mensaje te pregunta si has hecho `dropna()` o un `resample`), que las columnas coincidan en contenido y orden, que no haya ni un NaN (el calentamiento debe ser 0) y que solo aparezcan `-1`, `0` y `1`. Si todo está bien, convierte la tabla a `float`, así que no tienes que preocuparte del tipo. Además registra la función en `STRATEGIES`, y por eso el notebook 02 y el walk-forward pueden pedirla por su nombre con `get_strategy("sma_crossover")`. Sus tests están en `tests/test_plumbing.py` y ya pasan. La validación de parámetros al principio de cada estrategia (por ejemplo, `0 < fast < slow`) también está hecha: por eso `test_sma_crossover_rejects_fast_not_below_slow` pasa antes de que escribas nada.

Lo que el decorador no puede ver

Comprueba la forma de la salida, no su contenido: una estrategia que mira el futuro devuelve una tabla impecable de -1 , 0 y 1 . De eso se encargan los tests de causalidad.

Por qué una estrategia nunca hace shift

Recuerda la convención del motor (capítulo 02): la posición del día t es la señal del día anterior, $w_t = s_{\{t-1\}}$, y gana $r_t = P_t / P_{\{t-1\}} - 1$. Es decir: calculas la señal con el cierre de t , abres la posición a ese cierre (una idealización) y cobras lo que haga el activo del cierre de t al de $t+1$. Si la estrategia desplazara además su señal, el desfase sería doble y operarías un día tarde. No es sesgo de anticipación, pero estarías backtestando otra estrategia, y los tests lo detectan porque exigen que la señal aparezca el mismo día en que los datos la permiten. El desfase vive en un único sitio: `signals_to_positions`, con su parámetro `lag` (por defecto `config.SIGNAL_LAG`).

No confundas dos usos de `shift`. Desplazar la **señal de salida** está prohibido dentro de la estrategia. Desplazar **datos de entrada** para mirar al pasado es legítimo, y lo harás en el momentum: `shift(n)` con $n > 0$ trae a la fila actual el valor de n filas antes, que ya se conocía. Lo que nunca es legítimo es `shift(-n)`, que trae valores del futuro.

Operar en el mismo cierre es una aproximación

Calcular la señal con el cierre de t y operar a ese mismo cierre supone que puedes mandar la orden en el último instante. El README lo reconoce: `lag=2` (operar un día después) es la versión conservadora. Como el desfase vive solo en el motor, probarla no exige tocar ninguna estrategia.

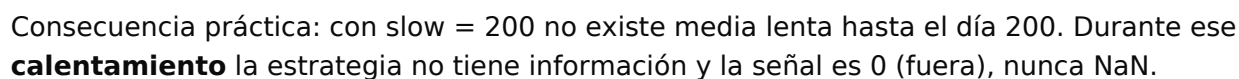
Causalidad: dos tests que vigilan el futuro

La regla «la señal de t solo usa precios hasta t » se comprueba de dos formas, para las seis combinaciones de estrategia y parámetros de la lista `CASES` de `tests/test_strategies.py`:

- `test_strategy_is_causal_truncating_the_future` calcula las señales con todo el histórico y con solo las primeras k filas ($k = 400, 523$ y 777). Las k primeras señales tienen que coincidir exactamente. Es la situación de quien opera en tiempo real: el día k no existen las filas siguientes.
- `test_strategy_is_causal_perturbing_the_future` multiplica todos los precios a partir de la fila 600 por factores aleatorios entre $0,5$ y $1,5$. Las señales anteriores a la fila 600 no pueden cambiar.

Entre los dos atrapan los errores clásicos: ventanas centradas (`center=True`), rellenos hacia atrás (`bfill`), `shift(-1)` y cualquier normalización con estadísticos de toda la muestra (dividir por el máximo histórico, restar la media de todo el periodo). Hay un error que solo atrapa el primero: tratar el último día de los datos como especial. Si la estrategia diera por cerrado el mes en el último día disponible, con los datos truncados a mitad de mes generaría una señal ese día; con el histórico completo, no. Perturbar precios no cambia la longitud de la serie, así que el segundo test no lo vería. Ese es el motivo de la regla del último día de `_month_end_dates` (sección 2). Y no es solo cosa de tests: el walk-forward (capítulo 05) le pasa a la estrategia únicamente los precios anteriores al final de cada tramo, y solo si es causal ve lo mismo que habría visto alguien operando en tiempo real.

Una media móvil resume dónde ha estado el precio últimamente. La rápida (pocos días) reacciona antes que la lenta (muchos días); si la rápida está por encima, el precio reciente es más alto que el de largo plazo: hay tendencia alcista. El cruce 50/200 es el más popular (el «cruce dorado» cuando la media de 50 supera a la de 200), y reglas de este tipo se estudian en la literatura académica al menos desde Brock, Lakonishok y LeBaron (1992). Su punto débil es el retraso: entra tarde, sale tarde y en mercados laterales da señales falsas que se pagan en costes.



Rebalancear una vez al mes significa calcular la señal solo en los fines de mes y mantenerla hasta el siguiente, pase lo que pase entre medias.



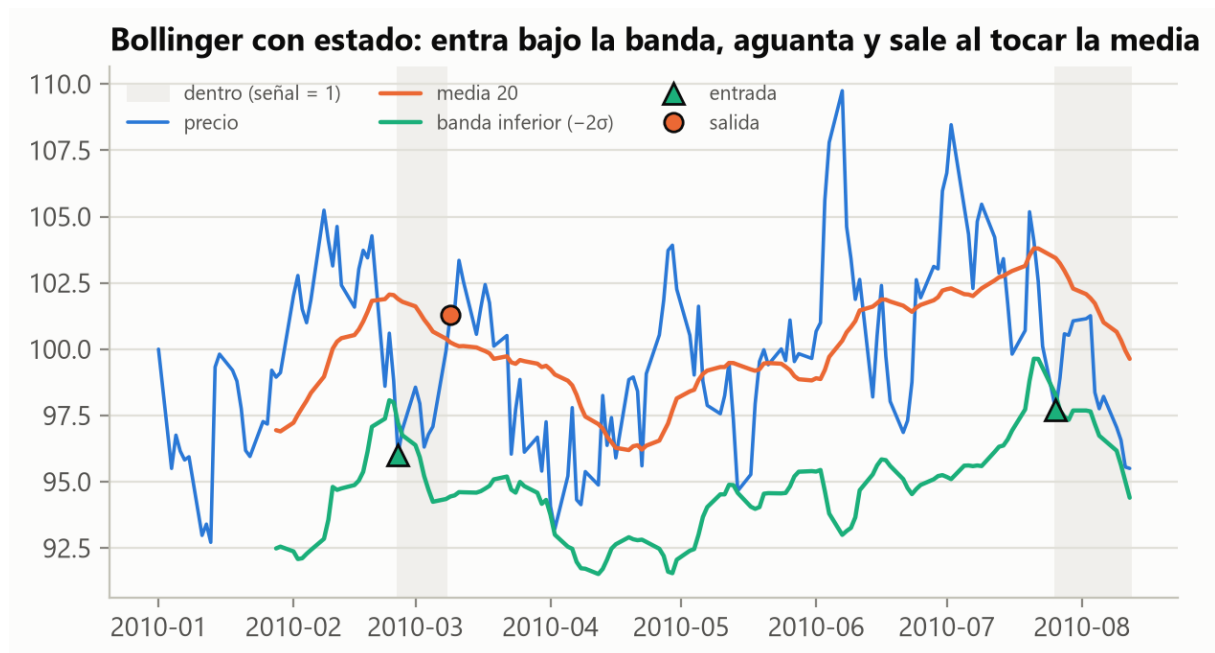
Precio diario con los fines de mes marcados. La señal, escalonada, solo puede cambiar en un fin de mes.

La variante **12-1** ignora el último mes: mide la rentabilidad desde hace 12 meses hasta hace 1. Viene del momentum de sección cruzada, donde es estándar porque a un mes vista los rendimientos tienden a revertir (Jegadeesh, 1990; Lehmann, 1990), en parte por efectos de microestructura como el rebote entre precio comprador y vendedor. Saltarse ese mes quita ruido a la señal. Moskowitz, Ooi y Pedersen usan los 12 meses completos; el parámetro `skip_months` te deja comparar ambas versiones en el notebook 02.

Apostar por la vuelta a la media: bandas de Bollinger

La hipótesis contraria: las desviaciones respecto a una media local son pasajeras. Las bandas de Bollinger (John Bollinger, años ochenta) dibujan una media móvil de N días y, a k desviaciones típicas por encima y por debajo, las bandas superior e inferior; los valores clásicos son $N = 20$ y $k = 2$. Un cierre por debajo de la banda inferior es un precio anormalmente bajo respecto a los últimos 20 días: se compra esperando que vuelva a la media y se vende cuando llega a ella (no a la banda superior: el objetivo es la media).

Aquí aparece algo nuevo: la estrategia tiene **estado**. El día siguiente a la entrada el precio puede estar ya por encima de la banda inferior sin haber llegado a la media. ¿Sigues dentro? Sí: entraste y aún no se ha cumplido la condición de salida. La señal de hoy depende de lo que pasó antes. Una regla sin memoria («largo si el cierre está bajo la banda») cerraría casi siempre al día siguiente; `test_bollinger_mean_reversion_is_stateful` la detecta buscando días con señal 1 y cierre por encima de la banda inferior.



Precio con su media y sus bandas. Se marcan las entradas (cierre bajo la banda inferior) y las salidas (cierre en la media o por encima); los periodos dentro de la posición están sombreados.

Con `allow_short=True` la estrategia es simétrica: entra corta cuando el cierre supera la banda superior y sale cuando cae a la media o por debajo.

¿Desviación muestral o poblacional?

Bollinger define las bandas con la desviación típica poblacional (dividiendo entre N). Este proyecto usa la muestral (entre $N - 1$), que es la que calcula pandas por defecto y la que esperan los tests. Con $N = 20$ la diferencia es un factor $\sqrt{(20/19)} \approx 1,026$: bandas un 2,6 % más anchas. Lo importante es ser coherente.

Vectorizar frente a bucles

Lo más obvio para una estrategia con estado es un bucle día a día con una variable que recuerda si estás dentro: es lo que hace `bollinger_reference` en `tests/helpers.py`. Es correcto pero lento, y en el walk-forward la estrategia se ejecuta cientos de veces. Las operaciones de pandas sobre tablas enteras corren en código compilado, decenas o cientos de veces más rápido, y además alinean por fecha. El precio es que no todo estado se vectoriza: uno que solo depende de «cuál fue el último evento» sí (sección 2); uno que depende de valores acumulados, como un *stop* a un 5 % del precio de entrada, suele necesitar un bucle o numba. La forma de trabajar de los tests es la buena: versión vectorizada, comparada con un bucle lento pero obvio (`test_bollinger_mean_reversion_matches_day_by_day_reference`).

2. Formalización

Todo se aplica columna a columna, así que basta con pensar en un activo. `P_t` es el cierre de la fila t ($t = 0, 1, 2, \dots$) y `s_t` la señal ($-1, 0$ o 1).

Media móvil simple y calentamiento

$$SMA_t(n) = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

Solo existe si hay n precios, es decir, desde la fila $t = n - 1$; antes no está definida (NaN). En pandas, `rolling(n, min_periods=n)` hace exactamente eso. Para ventanas de tamaño entero `min_periods` ya vale n por defecto, pero escríbelo: documenta la intención, y con ventanas temporales (por ejemplo "30D") el valor por defecto es 1, que daría medias de un solo precio.

```
x = pd.Series([1.0, 2.0, 3.0, 4.0])
x.rolling(3, min_periods=3).mean() # NaN, NaN, 2.0, 3.0
x.rolling(3, min_periods=1).mean() # 1.0, 1.5, 2.0, 3.0 <- medias con menos de 3 datos
```

Cruce de medias

Con `n_f < n_s` (ventana rápida y lenta), la versión con cortos es:

$$s_t = \text{sign}(SMA_t(n_f) - SMA_t(n_s))$$

Es decir: 1 si la rápida está por encima, -1 si está por debajo y 0 si coinciden. La versión solo largo se queda con la parte positiva:

$$s_t = \max(\text{sign}(SMA_t(n_f) - SMA_t(n_s)), 0)$$

Si alguna de las dos medias no existe, `s_t = 0`. Ejemplo con `n_f = 2` y `n_s = 3`:

Fila	Cierre	SMA(2)	SMA(3)	Solo largo	Con cortos
0	10	—	—	0	0
1	11	10,5	—	0	0
2	12	11,5	11,000	1	1
3	11	11,5	11,333	1	1
4	9	10,0	10,667	0	-1
5	8	8,5	9,333	0	-1

La fila 1 tiene media rápida pero no lenta: señal 0. La fila 2 es la primera con las dos medias y ya tiene señal: no hay desfase.

Momentum en fines de mes

Llama M al conjunto de fines de mes del índice y numera sus fechas $m = 0, 1, 2 \dots$ (la tabla mensual). Con `L = lookback_months` y `j = skip_months`:

$$\text{mom}_m = \frac{P_{m-j}}{P_{m-L}} - 1$$

Los subíndices cuentan fines de mes, no días. La señal mensual es `S_m = 1` si `mom_m > 0`; -1 si `mom_m < 0` y se permiten cortos; 0 en el resto, incluido cuando `mom_m` no existe (las primeras L filas de la tabla mensual). Con $L = 12$, la primera señal posible está en la fila 12, el 13.º fin de mes.

En el calendario diario, cada día hereda la señal del último fin de mes que no sea posterior a él. Si llamas `m*(t)` a ese fin de mes:

$$S_t = S_{m^*(t)}$$

y `s_t = 0` si todavía no ha habido ningún fin de mes. El propio fin de mes ya lleva la señal nueva: el desfase lo pone el motor.

Ejemplo con $L = 3$ sobre siete fines de mes, con cortos:

Mes	Cierre	mom con j = 0	Señal	mom con j = 1	Señal
1	100	—	0	—	0
2	104	—	0	—	0
3	101	—	0	—	0
4	107	$107/100 - 1 = +7,00 \%$	1	$101/100 - 1 = +1,00 \%$	1
5	103	$103/104 - 1 = -0,96 \%$	-1	$107/104 - 1 = +2,88 \%$	1
6	98	$98/101 - 1 = -2,97 \%$	-1	$103/101 - 1 = +1,98 \%$	1
7	99	$99/107 - 1 = -7,48 \%$	-1	$98/107 - 1 = -8,41 \%$	-1

En los meses 5 y 6 la caída reciente pesa mucho en la versión 3-0 y nada en la 3-1, que aún «no la ha visto»: saltarse un mes hace la señal más lenta.

Desplazar sobre la tabla mensual es desplazarse meses. `shift(n)` mueve n filas, sean lo que sean. Sobre la tabla diaria, `shift(12)` son 12 sesiones; sobre una tabla con una fila por fin de mes, son 12 meses:

```
cierres_mes = pd.Series(
    [100, 104, 101, 107, 103],
    index=pd.to_datetime(["2024-01-31", "2024-02-29", "2024-03-28", "2024-04-30", "2024-05-31"]),
)
cierres_mes.shift(2)  # NaN, NaN, 100, 104, 101 -> el valor de dos fines de mes antes
```

El 28-03-2024 es el último día con cotización de marzo (el 29 fue Viernes Santo): los fines de mes son días del índice, no fechas del calendario. Y no uses `shift(freq=...)`: con ese argumento se mueven las etiquetas de fecha, no los valores.

La regla del último día. `_month_end_dates` (ya hecha) marca como fin de mes el último día de cada mes presente en el índice: una fecha cuyo mes es distinto del de la fecha siguiente. El último día del índice no tiene fecha siguiente, así que se trata aparte: solo cuenta si es el último día hábil de su mes (`BMonthEnd`). `test_month_end_dates` lo resume: con datos del 1 de enero al 17 de marzo de 2020, los fines de mes son el 31 de enero y el 28 de febrero, porque marzo «no ha terminado»; con datos hasta el 31 de marzo, el 31 de marzo sí cuenta. Sin esta regla, el 17 de marzo pasaría por fin de mes y generaría una señal que desaparecería al llegar datos nuevos: justo lo que detecta el test de truncar el futuro.

Festivos

`BMonthEnd` conoce los fines de semana, no los festivos. El 31 de mayo de 2021 fue festivo en EE. UU., así que la última sesión de mayo fue el 28: con el histórico completo es fin de mes, pero con datos que acaban ese mismo día todavía no cuenta. No hay sesgo de anticipación (los festivos se conocen de antemano), pero en vivo actualizarías la señal un día más tarde que en la simulación. Los tests usan días hábiles sin festivos.

Bollinger como máquina de estados

Con $N = \text{window}$ y $k = \text{n_std}$:

$$m_t = \frac{1}{N} \sum_{i=0}^{N-1} P_{t-i}$$
$$\hat{\sigma}_t = \sqrt{\frac{1}{N-1} \sum_{i=0}^{N-1} (P_{t-i} - m_t)^2}$$
$$B_t^{\text{inf}} = m_t - k \cdot \hat{\sigma}_t$$
$$B_t^{\text{sup}} = m_t + k \cdot \hat{\sigma}_t$$

Define dos eventos para la pata larga: entrada, $E_t = 1$ si $P_t < B_t^{\text{inf}}$; salida, $X_t = 1$ si $P_t \geq m_t$ (los dos valen 0 si no se cumplen o si las bandas no existen). Como $B_t^{\text{inf}} \leq m_t$, nunca coinciden el mismo día. El estado largo Q_t (0 o 1) sigue esta recurrencia, con $Q_{-1} = 0$:

$$Q_t = E_t + (1 - E_t)(1 - X_t)Q_{t-1}$$

Léela así: si hoy hay entrada, 1; si hoy hay salida, 0; si no pasa nada, lo de ayer. La pata corta C_t (-1 o 0) es la simétrica, con entrada $E^c_t = 1$ si $P_t > B_t^{\text{sup}}$ y salida $X^c_t = 1$ si $P_t \leq m_t$:

$$C_t = -E^c_t + (1 - E^c_t)(1 - X^c_t)C_{t-1}$$
$$S_t = Q_t + C_t$$

¿Puede la suma valer 0 por estar largo y corto a la vez? No. Si $Q_t = 1$, o hoy hubo entrada ($P_t < B_t^{\text{inf}} \leq m_t$) o no hubo salida ($P_t < m_t$): en ambos casos $P_t < m_t$. Si $C_t = -1$, por el mismo razonamiento $P_t > m_t$. Las dos cosas no pueden darse el mismo día, y en los días sin bandas el estado se arrastra de uno en que tampoco se daban. Por tanto S_t solo puede valer -1, 0 o 1, y cuando la estrategia pasa de corto a largo lo hace de golpe, en un solo día.

Si falta un precio (NaN) dentro de una ventana, la media y la σ de esa ventana son NaN, las comparaciones dan False y no hay ni entrada ni salida: el estado se mantiene. Es lo mismo que hace el bucle de referencia.

Estados sin bucles: rellenar hacia delante

La recurrencia parece pedir un bucle, pero tiene una lectura que no lo necesita: **el estado de hoy es el valor del último evento ocurrido hasta hoy** (1 si fue una entrada, 0 si fue una salida), o 0 si aún no ha habido ninguno. Y «el último valor conocido hasta hoy» es justo lo que calcula un relleno hacia delante.

Míralo con un termostato que enciende la calefacción cuando la temperatura baja de 18 °C y la apaga cuando llega a 21 °C. Entre 18 y 21 no hace nada: mantiene lo que estuviera haciendo. Se resuelve en tres pasos: una columna de marcas que empieza vacía (NaN), con 1 donde hay orden de encender y 0 donde hay orden de apagar; un relleno hacia delante; y un 0 en lo que siga vacío (antes de la primera orden).

Hora	T (°C)	¿T < 18?	¿T ≥ 21?	Marcas	Tras rellenar	Estado	Sin memoria
0	20	no	no	NaN	NaN	0	0
1	17	sí	no	1	1	1	1
2	19	no	no	NaN	1	1	0
3	20	no	no	NaN	1	1	0
4	21	no	sí	0	0	0	0
5	19	no	no	NaN	0	0	0
6	17	sí	no	1	1	1	1
7	18	no	no	NaN	1	1	0

La última columna es la regla sin memoria («encendida si $T < 18$ »): apaga a las 2 aunque la casa siga fría. Los NaN no estorban, son la herramienta: significan «hoy no ha pasado nada, hereda». Si el termostato controlara además un aire acondicionado (-1 al superar 26°C , 0 al bajar a 23°C), harías otra columna de marcas con sus propias reglas y sumarías las dos. Es la misma idea que las dos patas de Bollinger.

Las piezas de pandas, por separado:

```
x = pd.Series([np.nan, 1.0, np.nan, np.nan, 0.0, np.nan])
x.fffll()          # NaN, 1, 1, 1, 0, 0   cada NaN toma el último valor no NaN anterior
x.fffll().ffllna(0) # 0, 1, 1, 1, 0, 0    lo que no tenía nada antes pasa a 0

y = pd.Series([3, 8, 5, 9])
y.mask(y > 6, -1)   # 3, -1, 5, -1   sustituye donde la condición es True
y.where(y > 6, -1)  # -1, 8, -1, 9   conserva donde la condición es True
```

Una tabla vacía del tamaño de los precios se crea con `pd.DataFrame(np.nan, index=..., columns=...)`. Como `fffll` solo mira hacia atrás, la técnica es causal por construcción.

Ejemplo a mano: Bollinger con ventana 3 y $k = 1$

Es el ejemplo de `test_bollinger_mean_reversion_hand_example` y de `test_bollinger_mean_reversion_hand_example_with_shorts` (`HAND_CLOSE`, `window=3`, `n_std=1`). Media y σ usan los tres últimos cierres (σ muestral, dividiendo entre $N - 1 = 2$). En la columna de eventos, «ent.» es entrada y «sal.» es salida; L es la pata larga y C la corta. Una salida de una pata que ya estaba fuera no cambia nada, pero se anota igual.

Fila	Cierre	Media	σ	Inferior	Superior	Eventos	Larga	Larga/corta
0	10	—	—	—	—	—	0	0
1	10	—	—	—	—	—	0	0
2	11	10,333	0,577	9,756	10,911	sal. L, ent. C	0	-1
3	10	10,333	0,577	9,756	10,911	sal. C	0	0
4	7	9,333	2,082	7,252	11,415	ent. L, sal. C	1	1
5	8	8,333	1,528	6,806	9,861	sal. C	1	1
6	8,6	7,867	0,808	7,058	8,675	sal. L	0	0
7	11	9,200	1,587	7,613	10,787	sal. L, ent. C	0	-1
8	12	10,533	1,747	8,786	12,281	sal. L	0	-1
9	10,5	11,167	0,764	10,403	11,930	sal. C	0	0
10	10	10,833	1,041	9,793	11,874	sal. C	0	0

Fila	Cierre	Media	σ	Inferior	Superior	Eventos	Larga	Larga/corta
11	14	11,500	2,179	9,321	13,679	sal. L, ent. C	0	-1
12	13	12,333	2,082	10,252	14,415	sal. L	0	-1
13	11	12,667	1,528	11,139	14,194	ent. L, sal. C	1	1
14	10	11,333	1,528	9,806	12,861	sal. C	1	1

Las filas clave:

- Filas 0 y 1: calentamiento. Con tres datos por ventana no hay bandas hasta la fila 2: señal 0.
- Fila 2: $11 > 10,911$, por encima de la banda superior: entrada corta (solo con cortos). En la fila 3, $10 \leq 10,333$: el precio vuelve a la media y el corto se cierra.
- Fila 4: $7 < 7,252$: entrada larga. El mismo cierre está bajo la media, así que si hubiera un corto abierto se cerraría.
- Fila 5: el caso que define una estrategia con estado. 8 no está bajo la banda inferior (6,806), así que una regla sin memoria daría 0; pero tampoco ha llegado a la media (8,333), así que no hay salida y el largo **se mantiene**.
- Fila 6: $8,6 \geq 7,867$: el precio alcanza la media y el largo se cierra. La media ha bajado mucho porque la ventana contiene el 7: la media a la que se vuelve es la de hoy, no la del día de entrada.
- Filas 7 y 8: entrada corta en $11 > 10,787$, y en la fila 8 se mantiene porque 12 no supera la nueva banda superior (12,281) pero tampoco ha caído a la media (10,533). Se cierra en la fila 9 ($10,5 \leq 11,167$).
- Filas 11 y 12: lo mismo. Entrada corta en $14 > 13,679$, y se mantiene con 13, que queda entre la media (12,333) y la banda superior (14,415).
- Fila 13: $11 < 11,139$ abre el largo y, a la vez, $11 \leq 12,667$ cierra el corto: la señal salta de -1 a 1 en un día (el motor cobrará un turnover de 2). En la fila 14, 10 no llega a la media (11,333): el largo sigue.

Comprueba que las dos últimas columnas coinciden con **HAND_LONG** y **HAND_LONG_SHORT**.

3. Del papel al código

TODO 3.1 · sma_crossover

Qué hace. Calcula, para cada activo, la media móvil rápida y la lenta y devuelve 1 donde la rápida está por encima de la lenta, 0 en el resto y, si `allow_short=True`, -1 donde está por debajo.

Firma. Lleva el decorador `@strategy`.

```
def sma_crossover(
    prices: pd.DataFrame, fast: int = 50, slow: int = 200, allow_short: bool = False
) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Las medias solo existen con el número completo de precios; mientras falte alguna, la señal es 0: `test_sma_crossover_trend_starts_when_slow_average_exists` exige 0 en las 49 primeras filas con `slow=50`.
- Sin desfase: en ese mismo test, con un activo que sube en línea recta, la señal vale 1 desde la fila 49, el primer día con media lenta (y -1 desde ahí para el que baja, con cortos).
- Comparaciones estrictas, de modo que medias iguales dan 0. `test_sma_crossover_hand_example` (cierres 1, 2, 3, 2, 1, 1, 2, 3 con medias de 2 y 3 días) comprueba los valores fila a fila, con y sin cortos.
- Sin cortos nunca aparece -1: `test_sma_crossover_long_only_never_short`.
- `fast >= slow` lanza `ValueError` (ya hecho): `test_sma_crossover_rejects_fast_not_below_slow`.
- Formato, «opera alguna vez» y causalidad: `test_strategy_output_format` y los dos `test_strategy_is_causal_*` con los ids `sma_crossover` y `sma_crossover-short` (`fast = 5`, `slow = 20`).

Pseudocódigo.

1. Para cada activo y cada día `t`, media rápida = media de los últimos `fast` cierres (incluido el de `t`). Si no hay `fast` cierres: "sin valor".
2. Lo mismo para la media lenta, con `slow` cierres.
3. Señal = 1 donde rápida > lenta; 0 en el resto (incluidos los días "sin valor").
4. Si `allow_short`: además, -1 donde rápida < lenta.
5. Devuelve una tabla con las mismas fechas y columnas que los precios.

Pistas.

Pista 1

No pienses en un activo ni en un día: una media móvil sobre un DataFrame se calcula para todas las columnas a la vez, y comparar dos DataFrames de la misma forma da una tabla de True/False del mismo tamaño.

Pista 2

`rolling(n, min_periods=n)` seguido de `.mean()` para cada media; los operadores `>` y `<` entre tablas; `astype(float)` para pasar de booleanos a 0.0 y 1.0. Para la versión con cortos, piensa en la diferencia de dos tablas de ceros y unos.

Pista 3

Una comparación con NaN da False, así que si construyes la señal a partir de comparaciones el calentamiento sale 0 sin hacer nada más. Si prefieres el signo de la diferencia de medias (`np.sign`), el signo de NaN es NaN: tendrás que rellenar con 0 y, en la versión solo largo, quedarte con la parte positiva. Nunca uses `dropna()`: las fechas de salida tienen que ser las de entrada.

TODO 3.2 · momentum_12m

Qué hace. En cada fin de mes calcula la rentabilidad entre hace `lookback_months` y hace `skip_months` fines de mes y fija la señal: 1 si es positiva; -1 si es negativa y se permiten cortos; 0 en el resto. Entre dos fines de mes mantiene la última señal.

Firma. Lleva el decorador `@strategy`.

```
def momentum_12m(
    prices: pd.DataFrame,
    lookback_months: int = 12,
    skip_months: int = 0,
    allow_short: bool = False,
) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- La señal solo cambia en fechas de `_month_end_dates`: `test_momentum_12m_changes_only_at_month_ends`.
- Calentamiento y primera señal: con `rise_then_crash` (sube en línea recta hasta el 31-01-2011 y el 1-02-2011 se desploma a 50), `test_momentum_12m_warmup_and_first_signal` exige 0 antes del 31-01-2011 (el 13.º fin de mes) y 1 desde ese mismo día hasta el 25-02-2011: el precio ya ha caído, pero no hay fin de mes.
- `test_momentum_12m_after_crash`: el 28-02-2011 el precio (50) está por debajo del de 12 fines de mes antes: -1 con cortos y 0 sin ellos, y así sigue.
- `test_momentum_12m_skip_months`: con `skip_months=1`, el 28-02-2011 se mira $P(\text{ene-2011}) / P(\text{feb-2010}) - 1 > 0$ y se sigue largo; el 31-03-2011 el numerador ya es $P(\text{feb-2011}) = 50$ y la señal cae a 0.
- Un momentum nulo o no calculable da 0.
- Formato y causalidad con los ids `momentum_12m` y `momentum_12m-short` (este último con `skip_months=1` y cortos). El de truncar el futuro falla si das por cerrado un mes que no ha terminado; `_month_end_dates` ya lo resuelve.

Pseudocódigo.

1. Obtén las fechas de fin de mes del índice con la utilidad ya hecha.
2. Construye la tabla mensual: los cierres de esas fechas, una fila por fin de mes.
3. En la tabla mensual, para cada fila m:
 numerador = cierre de `skip\_months` filas antes (la propia fila si skip = 0)
 denominador = cierre de `lookback\_months` filas antes
 mom = numerador / denominador - 1 ("sin valor" si falta alguno)
4. Señal mensual: 1 si mom > 0; -1 si mom < 0 y allow_short; 0 en el resto.
5. Lleva la señal mensual al calendario diario: en los fines de mes, su señal; en los demás días, "sin valor".
6. Cada día "sin valor" hereda la señal del último fin de mes anterior.
7. Los días anteriores al primer fin de mes siguen "sin valor": ponlos a 0.

Pistas.

Pista 1

Trabaja a dos frecuencias: decide en la tabla mensual (donde «12 filas atrás» son 12 meses) y solo al final vuelve al calendario diario. Si intentas hacerlo todo en la tabla diaria acabarás contando sesiones en vez de meses.

Pista 2

`prices.loc[fechas]` selecciona las filas de fin de mes; `shift(n)` sobre la tabla mensual desplaza meses; `reindex(prices.index)` devuelve una tabla al índice diario con NaN en las fechas que no tenía; `ffill()` y `fillna(0)` hacen el resto. No necesitas `resample`.

Pista 3

Convierte el momentum en señal (0, 1 o -1) en la tabla mensual, antes de volver al índice diario. Así un fin de mes sin momentum calculable da 0, como dice el docstring, en vez de heredar la señal del mes anterior al rellenar hacia delante. Y no desplaces la señal mensual: el propio fin de mes ya lleva la señal nueva (el test de la primera señal lo comprueba el 31-01-2011).

TODO 3.3 · bollinger_mean_reversion

Qué hace. Calcula la media y las bandas de Bollinger y mantiene un estado por activo: entra largo cuando el cierre cae bajo la banda inferior y sale cuando alcanza la media. Con `allow_short=True` añade la pata corta simétrica y devuelve la suma de las dos.

Firma. Lleva el decorador `@strategy`.

```
def bollinger_mean_reversion(
    prices: pd.DataFrame, window: int = 20, n_std: float = 2.0, allow_short: bool = False
) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Media y σ con exactamente `window` cierres; σ muestral (ddof = 1). En el calentamiento y antes de la primera entrada, 0.
- Desigualdades: entrada larga con `<` (estricta) y salida con `>=`; entrada corta con `>` y salida con `<=`.
- El ejemplo a mano de la sección 2, fila a fila: `test_bollinger_mean_reversion_hand_example` y `test_bollinger_mean_reversion_hand_example_with_shorts`.
- El mismo resultado que el bucle `bollinger_reference` sobre precios que revierten a la media (`ou_prices`), con (20, 2.0) y (10, 1.0), con y sin cortos:

`test_bollinger_mean_reversion_matches_day_by_day_reference` (cuatro casos).

- Hay estado: `test_bollinger_mean_reversion_is_stateful`.
- Formato y causalidad con los ids `bollinger_mean_reversion` y `bollinger_mean_reversion-short` (`window=10`, `n_std=1.5`).

Pseudocódigo.

1. Media y desviación típica muestral móviles de `window` cierres.
Banda inferior = media - n_std * sigma; banda superior = media + n_std * sigma.
2. Pata larga: tabla del tamaño de los precios, toda "sin valor".
Marca 1 donde cierre < banda inferior. Marca 0 donde cierre >= media.
3. Rellena cada "sin valor" con la última marca anterior; lo que siga
"sin valor" (antes de la primera marca) pasa a 0.
4. Si allow_short: pata corta en otra tabla, marcando -1 donde cierre > banda superior y 0 donde cierre <= media; rellena y pon a 0 lo que quede.
5. Señal = pata larga (+ pata corta si allow_short).

Pistas.

Pista 1

El estado solo cambia en los días con evento; entre eventos, se hereda. Representa «hoy no pasa nada» con NaN y deja que el relleno hacia delante haga de memoria, como en el termostato.

Pista 2

`rolling(window, min_periods=window)` con `.mean()` y `.std()` (que ya usa `ddof = 1`);
`pd.DataFrame(np.nan, index=..., columns=...)` para la tabla vacía; `mask` o `where` para escribir cada marca donde se cumple su condición; `ffill()` y `fillna(0)`.

Pista 3

Dentro de una pata, el orden en que escribes las marcas no importa: entrada y salida no pueden coincidir el mismo día. Lo que sí importa es no mezclar las patas en una sola tabla: en la fila 5 del ejemplo la salida corta ($8 \leq 8,333$) escribiría un 0 que cerraría el largo que debe mantenerse. Durante el calentamiento las comparaciones con NaN dan False, no se marca nada y todo acaba en 0. Crea la tabla vacía con NaN de tipo float para que todo el cálculo sea numérico.

4. Errores típicos

Desplazar la señal dentro de la estrategia

Un `shift(1)` al final «por seguridad». Síntoma: con el del motor, el desfase es de dos días. Lo detectan `test_sma_crossover_trend_starts_when_slow_average_exists` (la señal empieza en la fila 50 en vez de en la 49), `test_momentum_12m_warmup_and_first_signal` y los dos ejemplos a mano de Bollinger.

Medias con menos datos de la cuenta

`min_periods=1`, `expanding()` o una media exponencial sin calentamiento. Síntoma: señales en los primeros días, calculadas con dos o tres precios. Lo detecta `test_sma_crossover_trend_starts_when_slow_average_exists`.

Devolver NaN o recortar fechas

Dejar los NaN del calentamiento o quitarlos con `dropna()`. Síntoma: el decorador lanza `ValueError` antes de que se compruebe ningún valor.

Contar sesiones en vez de meses

Calcular el momentum con `shift(12)` o `shift(252)` sobre la tabla diaria. Síntoma: la señal cambia cualquier día, no solo a fin de mes. Lo detecta `test_momentum_12m_changes_only_at_month_ends`.

Fines de mes con resample

`resample("ME").last()` etiqueta cada mes con su último día natural (el 31-01-2010 era domingo), que puede no estar en el índice: al volver al calendario diario esas señales se pierden. Además crea una fila para un mes sin cerrar. Lo detectan `test_momentum_12m_warmup_and_first_signal` y `test_strategy_is_causal_truncating_the_future`. (El alias "M" ya no existe en pandas 3.) Usa `_month_end_dates`.

Bollinger sin memoria

Señal = «cierre bajo la banda inferior». Síntoma: las filas 5 y 14 del ejemplo salen 0 en vez de 1. Lo detectan `test_bollinger_mean_reversion_hand_example`, `test_bollinger_mean_reversion_is_stateful` y la comparación con el bucle de referencia.

Desviación poblacional

`np.std(ddof = 0 por defecto)` dentro de un `rolling(...).apply`. Las bandas salen más estrechas: en el ejemplo, en la fila 9 el 10,5 quedaría bajo una banda inferior de 10,543 (largo indebido) y en la fila 6 el 8,6 superaría una superior de 8,527 (corto indebido). Lo detectan los dos ejemplos a mano y `test_bollinger_mean_reversion_matches_day_by_day_reference`.

Las dos patas en una sola tabla

Escribir 1, -1 y 0 en la misma tabla de marcas. Síntoma: la salida de una pata borra la posición de la otra (filas 5, 8, 12 y 14 del ejemplo) y, si una entra el día en que la otra sale (filas 2, 4, 7, 11 y 13), gana la última marca escrita. Lo detecta `test_bollinger_mean_reversion_hand_example_with_shorts`.

Mirar el futuro sin querer

`rolling(..., center=True)`, `bfill()`, `shift(-1)` o normalizar con la media o el máximo de toda la serie. Síntoma: resultados demasiado buenos. Lo detectan `test_strategy_is_causal_truncating_the_future` y `test_strategy_is_causal_perturbing_the_future`.

Trampas de pandas 3

La asignación encadenada (`tabla[col][condicion] = 1`) modifica una copia temporal por el copy-on-write: la tabla original no cambia, la estrategia no opera nunca y falla `test_strategy_output_format`. Usa `mask`, `where` o `.loc[...]` en un solo paso. `fillna(method="ffill")` ya no existe (`TypeError`): usa `ffill()`. Y `pyproject.toml` convierte en error cualquier `FutureWarning` o `DeprecationWarning` emitido desde `src/`: una API obsoleta hace fallar el test aunque el resultado sea correcto.

5. Preguntas y ejercicios

1. ¿Por qué una estrategia devuelve señales y no posiciones? ¿Qué se perdería si cada estrategia aplicara su propio desfase?
2. Con `fast=50` y `slow=200`, ¿cuántas filas de calentamiento hay y en qué fila aparece la primera señal posible?
3. Explica por qué tratar el último día de los datos como fin de mes rompe el test de truncar el futuro pero no el de perturbarlo.
4. Con `rise_then_crash` y momentum 12-0, ¿qué señal hay el 15-02-2011, con el precio ya en 50? ¿Por qué no es un error de la estrategia?
5. En la tabla de momentum de la sección 2, ¿por qué la versión 3-1 tarda dos meses más en ponerse corta? ¿Qué ganas y qué pierdes al saltarte el último mes?
6. Razona que en Bollinger la pata larga y la corta nunca están activas a la vez. ¿Seguiría siendo cierto si el largo saliera al tocar la banda superior en lugar de la media?
7. Si subes `n_std` de 2 a 2,5, ¿qué esperas que pase con el número de operaciones, el tiempo invertido y el turnover? ¿Y si bajas `window` de 20 a 10?
8. Ejercicio (notebook 02, sección 1): para cada estrategia y activo, cuenta cuántas veces cambia la señal por año y la duración media de cada posición. Relaciónalo con la columna «Turnover anual» de la tabla comparativa.
9. Ejercicio (notebook 02): mide con `%timeit` tu Bollinger vectorizado frente a `bollinger_reference` de `tests/helpers.py` sobre un activo real y comprueba con `assert_series_equal` que dan lo mismo. Después completa la celda de variantes de la sección 6 (12-0 frente a 12-1, solo largo frente a largo/corto).

6. Checkpoint

```
python -m pytest tests/test_strategies.py
python -m pytest tests/test_strategies.py -rs
python -m pytest -k sma_crossover
python -m pytest -k momentum_12m
python -m pytest -k bollinger
```

El `conftest.py` convierte en *skipped* cualquier test que se tope con un `NotImplementedError`, con el motivo «TODO pendiente: TODO 3.x · ...», y al final lista los TODO pendientes y cuántos tests esperan a cada uno. `-rs` muestra el motivo de cada test saltado; `--todo-fail` hace que los pendientes cuenten como fallos. Un fallo de verdad (F) significa que tu código se ejecuta pero da otro resultado: el mensaje del `assert` dice qué convención no se cumple.

`tests/test_strategies.py` tiene 33 tests. Antes de empezar verás 1 passed (`test_sma_crossover_rejects_fast_not_below_slow`: la validación ya está hecha) y 32 skipped. Cada estrategia aporta sus tests más los seis de formato y causalidad de sus dos casos: tras el TODO 3.1, 10 passed y 23 skipped; tras el 3.2, 20 passed y 13 skipped; tras el 3.3, 33 passed. `-k` selecciona por nombre e id del caso: `-k bollinger` lanza los 13 de Bollinger. Otros ficheros también usan tus estrategias: `test_quantstats_check.py` backtestea un cruce de medias y `test_walkforward.py` optimiza el cruce y Bollinger.